



Launch a Kubernetes Cluster

JJ

jjoseph1608@outlook.com

Nodes (3) [Info](#)

< 1 >

Node name	Instance type	Compute	Managed by	Created	Status	CPU usage
ip-192-168-28-133.eu-west-2.compute.internal	t3.micro	Node group	nextwork-nodegroup	9 minutes ago	Ready	17%
ip-192-168-38-175.eu-west-2.compute.internal	t3.micro	Node group	nextwork-nodegroup	9 minutes ago	Ready	22%
ip-192-168-82-244.eu-west-2.compute.internal	t3.micro	Node group	nextwork-nodegroup	9 minutes ago	Ready	17%

Node groups (1) [Info](#)

EditDeleteAdd

< 1 >

Group name	Desired size	AMI release version	Launch template	Status
nextwork-nodegroup	3	1.33.11-20260529	eksctl-nextwork-eks-cluster-nodegroup-nextwork-nodegroup (1)	Active



jjoseph1608@outlook....

NextWork Student

nextwork.org

Introducing Today's Project!

In this project, I will launch an EC2 instance, create a Kubernetes cluster, and test how it handles failures, because I need to learn how Kubernetes works and how to build resilient systems that keep running when things go wrong



What is Kubernetes?

Kubernetes is a tool that automatically manages containers running across multiple servers. It handles deploying your app, keeping it running, scaling it up when you get more traffic, and scaling it back down when you don't need the extra resources. Companies and developers use Kubernetes because manually running containers across dozens or hundreds of servers gets out of hand fast. Kubernetes takes the pain out of it. You tell it what you want to run and how many copies you need, and it figures out where to put them, keeps them healthy, and handles updates without downtime. It also lets you treat your entire server fleet like one big computer instead of managing each one separately

I used `eksctl` to create my entire EKS cluster from the command line. The `create cluster` command I ran defined the cluster name, the Kubernetes version I wanted, the region it should run in, and how many worker nodes to spin up. `Eksctl` took all those parameters and built out everything automatically, including the VPC, subnets, security groups, and IAM roles that the cluster needed to function. Without `eksctl` I would've had to set up all of that manually, which would've taken hours.

I initially ran into two errors while using `eksctl`. The first one was because `eksctl` wasn't installed on my EC2 instance yet. When I tried to run the `create cluster` command, the terminal told me the command didn't exist. That's when I installed `eksctl`. The second one was because my EC2 instance didn't have the right IAM permissions to create the EKS cluster. Even after `eksctl` was installed, it tried to create VPCs, security groups, and other resources but didn't have permission to do it. I had to attach an IAM policy to my instance's role that gave it permission to perform all those actions on AWS. Once I did that, `eksctl` could finally build the cluster



NextWork Student

nextwork.org

```
t      #  
----- #####  
--\    \#####  
--     \#####  
--      \#/  
--       V- -> https://aws.amazon.com/linux/amazon-linux-2023  
-----  
--_ _ _  
-- /' /  
-- m/
```

```
[ec2-user@ip-172-31-8-172 ~]$ eksctl create cluster \  
--name network-eks-cluster \  
--nodegroup-name network-nodegroup \  
--node-type t3.micro \  
--nodes 3 \  
--nodes-min 1 \  
--nodes-max 3 \  
--version 1.33  
-bash: eksctl: command not found  
[ec2-user@ip-172-31-8-172 ~]$
```



eksctl and CloudFormation

CloudFormation helped create my EKS cluster because when I ran the `eksctl` command, it generated a CloudFormation template that defined all the infrastructure the cluster needed. CloudFormation then took that template and automatically built everything, which saved me from having to manually create each resource one by one. It created VPC resources because a Kubernetes cluster needs networking to run. CloudFormation built the VPC, subnets, route tables, security groups, and NAT gateways that the cluster sits on. It also created the IAM roles the cluster needs to talk to other AWS services. Essentially, CloudFormation handled all the infrastructure plumbing so I could just run one command and have a working cluster

There was a second CloudFormation stack for the node group. When you run `eksctl create cluster`, it actually creates two separate stacks: one for the cluster itself and one for the worker nodes. The difference between a cluster and node group is this. The cluster is the Kubernetes control plane, the part that manages and schedules your containers. It handles decisions about where things run, keeps everything healthy, and responds to your commands. The node group is the actual compute resources, the EC2 instances where your containers actually execute. You can have one cluster with multiple node groups running different types of instances if you want. So the first stack created your EKS cluster control plane. The second stack created the three `t3.micro` instances that will run your workloads. That's why you saw two stacks in CloudFormation



View nested

< 1 > ⚙

Stacks

☐	eksctl-nextwork-eks-cluster-nodegroup-nextwork-nodegroup 2026-06-14 03:03:37 UTC+0100 ✔ CREATE_COMPLETE
☒	eksctl-nextwork-eks-cluster-cluster 2026-06-14 02:53:33 UTC+0100 ✔ CREATE_COMPLETE

Latest dep

This is a timeli
reset.

🔍 Search e

IngressNod



The EKS console

I had to create an IAM access entry in order to give myself permission to actually use the Kubernetes cluster. Without it, the cluster wouldn't recognize me as someone allowed to run commands or deploy anything to it. An access entry is a rule that connects your AWS identity to the Kubernetes cluster and says what you're allowed to do. It's the bridge between AWS IAM and Kubernetes permissions. Instead of logging in with a password, you authenticate using your AWS credentials, and the access entry tells the cluster whether you should be let in. I set it up by going to the EKS console, finding my cluster, navigating to the Access tab, and adding a new entry with my AWS user or role. I gave myself admin permissions so I could run any command against the cluster. Once that was saved, I could start using kubectl to interact with the cluster

It took me about 15 minutes to create my cluster. Next time I could speed this up by saving the eksctl command as a script so I don't have to type it all out again. I could also prepare my IAM permissions and access entries ahead of time instead of setting them up after the cluster is already running. That way when I run the command everything is ready to go and I won't hit permission errors that slow me down



Nodes (3)

Info

Q Filter Nodes by property or value

< 1 >

Node name	Instance type	Compute	Managed by	Created	Status	CPU usage
ip-192-168-28-133.eu-west-2.compute.internal	t3.micro	Node group	nextwork-nodegroup	9 minutes ago	Ready	17%
ip-192-168-38-175.eu-west-2.compute.internal	t3.micro	Node group	nextwork-nodegroup	9 minutes ago	Ready	22%
ip-192-168-82-244.eu-west-2.compute.internal	t3.micro	Node group	nextwork-nodegroup	9 minutes ago	Ready	17%

Node groups (1)

Info

Q Filter node groups by property or value

< 1 >

Edit

Delete

Add

Node groups implement basic compute scaling through EC2 Auto Scaling groups.

Group name	Desired size	AMI release version	Launch template	Status
nextwork-nodegroup	3	1.33.11-20260529	eksctl-nextwork-eks-cluster-nodegroup-nextwork-nodegroup (1)	Active



EXTRA: Deleting nodes

EKS nodes are just EC2 instances. When you create your cluster, eksctl launches regular EC2 instances to serve as your worker nodes. That's why you see them in the EC2 console. The nodes running your Kubernetes workloads are the same compute resources you'd manage directly if you weren't using EKS. The difference is that EKS controls them for you instead of you managing each one manually

Desired size means the number of nodes you're running right now. You set it to 3, so your cluster starts with 3 worker nodes. Minimum and maximum sizes are helpful for auto-scaling. Your cluster can grow or shrink automatically based on how much work it needs to do. The minimum is the lowest you let it go (1 node in your case) and the maximum is the highest (3 nodes). If traffic spikes and you need more capacity, Kubernetes launches new nodes up to your maximum. When traffic drops, it shuts down nodes but won't go below your minimum. This saves you money because you only pay for what you're using, and you don't have to manually adjust the cluster size yourself

When I deleted my EC2 instances, Kubernetes noticed the nodes were gone and automatically rescheduled the containers that were running on them to the remaining nodes. This happened because Kubernetes wants to keep your workloads running no matter what. When a node disappears, the control plane detects it and moves those containers elsewhere so your app stays up. This is actually useful to test. It shows how Kubernetes handles failures. In real life, nodes go down all the time. Hardware fails. Updates happen. Kubernetes takes that in stride and just redistributes the work. That's why people use it



Instances (7) Info

Last updated less than a minute ago

Connect

Instance state

Actions

Launch instances

Saved filter sets

Choose filter set

Find Instance by attribute or tag (case-sensitive)

☐	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone
☐	nextwork-eks-cluster-nextwor...	i-048c4742d441694d0	Terminated	t3.micro	-	eu-west-2c
☐	nextwork-eks-instance	i-05f439fc654524b3f	Running	t3.micro	3/3 checks passec	eu-west-2c
☐	nextwork-eks-cluster-nextwor...	i-0ee01f3e1eeddbd73	Running	t3.micro	Initializing	eu-west-2c
☐	nextwork-eks-cluster-nextwor...	i-07902c87f7038ed54	Running	t3.micro	Initializing	eu-west-2a
☐	nextwork-eks-cluster-nextwor...	i-0a5713a707ee13060	Terminated	t3.micro	-	eu-west-2a
☐	nextwork-eks-cluster-nextwor...	i-0a57706c6c559a6ac	Terminated	t3.micro	-	eu-west-2b
☐	nextwork-eks-cluster-nextwor...	i-0a2318e1ae2f313ee	Running	t3.micro	3/3 checks passec	eu-west-2b

Select an instance



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

